

NxtWise Recommendation Studio

Explainable Product Recommendation System

Project Documentation

Dataset: Amazon Retail Demo Store (Synthetic E-Commerce Dataset)

Stack: Python · Flask · MySQL · scikit-learn · React · Vite

Document prepared: September 13, 2026

Index

Index	2
1. Problem Statement	4
1.1 Reviewability Requirements	4
2. Approach Followed	4
2.1 Data Governance	4
2.2 Architecture Boundary	4
2.3 Modelling Approach	4
2.4 Cold-Start Routing	5
2.5 Application Layer	5
2.6 Verification Approach	5
3. Technologies / Libraries Used	5
3.1 Core Stack	5
3.2 Notable Library Roles	6
4. Steps to Run the Project	6
4.1 Prerequisites	6
4.2 Database Setup	6
4.3 Data Preprocessing	6
4.4 Schema Creation and Data Loading	7
4.5 Starting the Application	7
4.6 Running the Automated Test Suite	7
4.7 Reviewer Demonstration Sequence	7
5. Key Assumptions	7
5.1 Data and Preprocessing Assumptions	7
5.2 Modelling Assumptions	8
5.3 Evaluation Assumptions	8
5.4 Application Assumptions	8
6. Results / Output	8
6.1 Held-Out Test Ranking Metrics	8
6.2 Robustness Audit Outcomes	9
6.3 Application and Test Verification Outcomes	9
6.4 Functional Output	9

7. Challenges Faced

10

7.1 Known Non-Blocking Issue

11

8. Possible Future Improvements

11

References

11

1. Problem Statement

An e-commerce catalog is too large for a shopper to browse efficiently, and the central product question addressed by this project is: given a shopper's prior behavior, which unseen products should appear in their next Top-10 recommendation list? The system must also address the practical case where no behavioral history exists for a shopper, without incorrectly presenting generic recommendations as personal.

The objective of the assessment was to build a practical, explainable, and reproducible hybrid product recommender covering preprocessing, user and product profiling, a popularity baseline, item-based collaborative filtering, content-based filtering, hybrid Top-N ranking, seen-item filtering, cold-start handling, a REST API, a React user interface, evaluation, testing, and documentation.

1.1 Reviewability Requirements

Beyond ranking quality, the project was additionally required to demonstrate:

- Reproducible raw-to-processed data preparation.
- A transparent baseline and multiple personalized recommendation methods.
- A leakage-safe (temporally correct) evaluation design.
- A usable application interface rather than only offline notebooks or scripts.
- Honest handling of new users who have no behavioral history.
- API and UI behavior that can be demonstrated live and verified with automated tests.

The project does not claim that an offline ranking metric is a guaranteed production conversion lift; the results establish comparative ranking quality on a synthetic held-out event population only.

2. Approach Followed

The project was delivered in fourteen sequential phases, each building on validated outputs of the previous phase. The approach deliberately separates offline data-science work from the online application, so that every reported number is traceable to a repeatable, scripted command.

2.1 Data Governance

The only permitted data source is the Amazon Retail Demo Store synthetic e-commerce dataset (interactions.csv, items.csv, users.csv). Raw files are never edited. A reproducible preprocessing pipeline validates required columns, identifiers, event types, and timestamps; it fails the run rather than silently dropping any interaction that references a user or item absent from the validated catalogs. All 6,000 users, 2,465 items, and 675,004 raw events passed validation with no rows removed.

2.2 Architecture Boundary

Offline scripts under `src/` are the only code permitted to read raw CSV files, validate schemas, produce processed datasets, build sparse representations, train recommendation artifacts, and calculate evaluation metrics. The online Flask application under `backend/app/` does not retrain a model, alter interaction data, or calculate metrics; at startup it loads already-selected artifacts, and for each request it validates parameters, asks the appropriate artifact for ranked item IDs, excludes previously seen items, hydrates catalog records through parameterized MySQL queries, and returns a stable JSON contract. MySQL stores only relational application entities (users, products, interactions); sparse matrices, nearest-neighbor structures, TF-IDF vectors, and Joblib artifacts remain local files.

2.3 Modelling Approach

Four recommendation approaches were implemented and evaluated under an identical global-temporal protocol:

- **Popularity baseline** — ranks catalog items by the number of unique training users who interacted with each item; used as a comparator and as the cold-start fallback.
- **Item-based collaborative filtering (CF)** — represents an interaction as binary and computes item-item cosine similarity; at serving time it sums a candidate's precomputed similarities to a user's historical items, storing only the top 100 positive neighbors per item.
- **Content-based filtering** — builds a TF-IDF document per product from its verified name, description, category levels, and gender, and scores candidates against an L2-normalized sum of a user's historical product vectors.
- **Validation-selected hybrid** — unions the top 100 CF and content candidates, independently max-normalizes each user's CF and content scores, and searches CF weights from 0.00 to 1.00 in 0.05 steps on the validation split only, selecting the weight with the best validation NDCG@10 (ties broken by recall@10, precision@10, then greater CF weight).

Validation selected a CF weight of 1.00 and a content weight of 0.00. The deployed hybrid route therefore serves the same output as pure collaborative filtering; this was an evidence-based outcome of the weight search, not a design assumption.

2.4 Cold-Start Routing

A serving router applies an explicit policy: a known user with personalized candidates is routed to the validation-selected CF-only hybrid; an anonymous, unknown, or candidate-less user is routed to global popularity. Seen-item filtering is enforced on every route, and each response carries a machine-readable provenance label (personalized_cf or popularity_cold_start) so the API and UI never present a non-personalized result as personal.

2.5 Application Layer

A Flask application factory exposes validated REST routes, a recommendation service, and a parameterized MySQL repository. A React/Vite interface allows a reviewer to pick a real dataset user (or create a temporary, non-persisted demo user), compare CF, content, and hybrid outputs side by side, inspect the user's most recent recorded activity, and see an explicit route label distinguishing personalized output from a cold-start fallback.

2.6 Verification Approach

Each phase added targeted unit and behavior tests. The final phase added an opt-in, non-mocked live integration test that exercises the real Flask application, the configured MySQL database, and the saved model artifacts together, covering health, user lookup, all three algorithms, catalog hydration, activity, cold start, and invalid-request handling.

3. Technologies / Libraries Used

The stack was intentionally kept monolithic and simple; deep learning, vector databases, message queues, and microservices were explicitly excluded as unnecessary infrastructure for this problem.

3.1 Core Stack

Layer	Technology
Programming language	Python 3.14
Offline data science / ML	Pandas, NumPy, SciPy, scikit-learn, Joblib
Backend web framework	Flask (application factory pattern)
Database	MySQL 8.0.16 or later (InnoDB, utf8mb4)
Frontend framework / tooling	React with Vite

Layer	Technology
Testing framework	Pytest

3.2 Notable Library Roles

- **scikit-learn** — TF-IDF vectorization and cosine-similarity based content representation.
- **SciPy / NumPy** — sparse matrix construction and normalized item-matrix multiplication for scalable item-based collaborative filtering.
- **Joblib** — serialization and loading of trained popularity, CF, content, hybrid, and cold-start router artifacts.
- **Pandas** — CSV ingestion, validation, transformation, and aggregation during preprocessing.
- **Flask** — REST API routing, request validation, and a safe, structured JSON error contract.
- **MySQL connector / parameterized queries** — transactional, batched upserts and read-only catalog hydration.
- **React + Vite** — reviewer-facing single-page interface for user selection, model comparison, and activity display.
- **Pytest** — preprocessing, database-loader, model, evaluation, API contract, and live integration test coverage.

4. Steps to Run the Project

4.1 Prerequisites

- Python 3.14 environment with the dependencies listed in requirements.txt.
- Node.js and npm for the React/Vite user interface.
- A running MySQL 8.0.16+ server, required because the schema uses enforced CHECK constraints in addition to foreign keys and unique indexes.
- A valid project-root .env file with non-placeholder MYSQL_HOST, MYSQL_PORT, MYSQL_DATABASE, MYSQL_USER, and MYSQL_PASSWORD values.
- Saved model artifacts present in the models/ directory (empty or hand-created artifact files must not be substituted).

4.2 Database Setup

Create the application database once, using a local MySQL administrator account:

```
CREATE DATABASE nxtwise_recommender CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Optionally, create a least-privilege application account instead of using the administrator account:

```
CREATE USER 'nxtwise_app'@'localhost' IDENTIFIED BY 'choose-a-strong-local-password';
GRANT ALL PRIVILEGES ON nxtwise_recommender.* TO 'nxtwise_app'@'localhost';
FLUSH PRIVILEGES;
```

Copy .env.example to a project-root .env file (never committed) and set the connection values:

```
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_DATABASE=nxtwise_recommender
MYSQL_USER=nxtwise_app
MYSQL_PASSWORD=your-local-password
```

4.3 Data Preprocessing

Run the reproducible pipeline against the frozen raw dataset:

```
python src/preprocessing.py --raw-dir data\raw\amazon_retail_demo --output-dir data\processed
```

4.4 Schema Creation and Data Loading

The schema does not need to be created manually; the loader creates the tables from backend/app/db/schema.sql, loads the processed files transactionally, and verifies row counts and foreign-key integrity:

```
python backend/app/db/loader.py --processed-dir data\processed
```

Expected load counts are 6,000 users, 2,465 products, and 675,004 interactions. Re-running the loader is safe because it uses parameterized, natural-key upserts.

4.5 Starting the Application

- Ensure the MySQL service is running and the project-root .env holds valid credentials.
- Start the Flask API (served at http://127.0.0.1:5000).
- Start the Vite development server for the React UI (served at http://127.0.0.1:5173).
- Open the UI in a browser at http://127.0.0.1:5173 or http://localhost:5173 (both origins are accepted by the API's CORS configuration).

4.6 Running the Automated Test Suite

The fast, self-contained backend suite:

```
python -m pytest backend/tests -q
```

The opt-in, non-mocked live API integration test, which requires a running MySQL instance and saved model artifacts:

```
$env:RUN_LIVE_API_TESTS = "1"
python -m pytest backend\tests\test_api_integration.py -q
```

4.7 Reviewer Demonstration Sequence

- Select a real dataset user and show their latest recorded activity.
- Run collaborative filtering, then content-based, then hybrid for the same user, and observe the route label and product cards.
- Explain that hybrid intentionally routes to CF because validation selected a CF weight of 1.00.
- Create a temporary demo user and show the labeled popularity cold-start result.
- Optionally run the automated verification commands.

5. Key Assumptions

5.1 Data and Preprocessing Assumptions

- The full 675,004-row interaction log requires no sampling: 42.5 MB of input, 6,000 users, and 2,465 items, with no referential gaps, made subsetting unnecessary.
- All five verified event types (View, AddToCart, ViewCart, StartCheckout, Purchase) are preserved as dataset facts; no relative event-strength weighting is invented during preprocessing.
- The DISCOUNT field is treated as categorical Yes/No rather than numeric, based on inspection of the raw values.
- A missing PROMOTED value is normalized to "unknown" rather than inferred as false, because 1,856 of 2,465 items have no explicit value and inferring false would be an unsupported assumption.
- Any interaction referencing a user or item absent from the validated catalogs is treated as a pipeline failure condition rather than a row to silently drop.

5.2 Modelling Assumptions

- Collaborative filtering treats every validated interaction type as a binary behavioral signal; it does not treat Purchase as the sole or a specially weighted signal.
- Item-item similarity is computed only from co-interaction patterns; only the top 100 positive-scoring neighbors are stored per item, bounding candidate generation.
- Content representation is restricted to verified catalog fields (name, description, category level 1, category level 2, gender); price and promotion are excluded because their value as recommendation signals has not been established by experiment.
- The hybrid blend weight is chosen exclusively on the validation split, never on the held-out test split, to avoid leakage into model selection.
- A user with no personalized candidates is assumed to be better served by global popularity than by a fabricated personalized result.

5.3 Evaluation Assumptions

- A global temporal split (rather than a random split) is assumed to be the only leakage-safe evaluation design for behavioral event data.
- An evaluation user is eligible only if they have training-window history and at least one unseen item in the corresponding holdout window.
- Previously seen items are assumed to be correctly excluded from both the recommendation candidates and the holdout relevance set, since the product requirement is that seen items are never recommended.
- No relevance metric is assumed or fabricated for a genuinely new user, since the frozen dataset contains no later interactions for such a user.

5.4 Application Assumptions

- MySQL is assumed to be the correct store for relational application entities only; sparse matrices, similarity structures, and TF-IDF vectors are assumed unsuitable for a transactional database and are kept as local files.
- A temporary demo user is assumed to be more honest for a cold-start demonstration than inserting a synthetic user or synthetic interaction history into MySQL.
- Local development is assumed to originate only from the Vite default origins (<http://localhost:5173> and <http://127.0.0.1:5173>), which are the only origins allowed by CORS by default.
- Product images are assumed to be out of scope because the frozen catalog does not establish a reliable local image mapping.

6. Results / Output

6.1 Held-Out Test Ranking Metrics

All models were evaluated on an identical, leakage-safe temporal split (472,504 training events, 101,250 validation events, 101,250 test events). Precision@10, Recall@10, and NDCG@10 are Top-N ranking metrics, not classification accuracy.

Model	Test Precision@10	Test Recall@10	Test NDCG@10
Popularity (baseline)	0.004352	0.041758	0.018441
Content-based (TF-IDF)	0.003912	0.036557	0.026511

Model	Test Precision@10	Test Recall@10	Test NDCG@10
Collaborative filtering	0.066110	0.549275	0.354436
Validation-selected hybrid	0.066110	0.549275	0.354436

The collaborative filtering model recovers on average about 54.9% of an eligible user's later unseen relevant items within a Top-10 list. Its NDCG@10 bootstrap 95% confidence interval is [0.339695, 0.369477]. The paired per-user NDCG@10 difference between the selected hybrid and popularity is 0.335995, with a 95% bootstrap confidence interval of [0.322371, 0.349737]; since this interval remains entirely above zero, the collaborative-filtering advantage is robust for this held-out test population.

6.2 Robustness Audit Outcomes

Model	Catalog coverage	Category diversity
Popularity	0.61%	0.9113
Content-based	77.36%	0.5722
Collaborative filtering / selected hybrid	99.55%	0.8717

Catalog coverage is the fraction of the 2,465 catalog items that appeared at least once across all test Top-10 lists; category diversity is the mean within-list pairwise category-level-2 diversity, both reported descriptively rather than as optimization targets. Performance across pre-test history-length slices was strongest for users with the longest history (13-15 items: NDCG@10 0.489905), consistent with a collaborative model benefiting from more behavioral signal.

6.3 Application and Test Verification Outcomes

Check	Result
Backend test suite with live integration enabled	41 passed
Source compilation (compileall)	Passed
Real Flask API smoke check (health, CF, content, hybrid, cold start, activity)	Passed
React production build (npm run build)	Passed
npm audit --omit=dev	0 vulnerabilities
Fresh-process Joblib model/router artifact loading	Passed
git diff --check	Passed

Live database verification against the configured MySQL instance returned exactly 6,000 users, 2,465 products, and 675,004 interactions with no orphan foreign-key records. The live API check returned HTTP 200 for health plus collaborative filtering, content, hybrid, and cold-start recommendation requests, and the UI walkthrough rendered both personalized cards and a labeled cold-start fallback.

6.4 Functional Output

- A Flask REST API exposing health, user lookup/search, recent activity, demo-user validation, and a single recommendation endpoint supporting the cf, content, and hybrid algorithms.
- A React/Vite reviewer interface supporting dataset-user selection, model comparison, a visible route label (PERSONALIZED or COLD START), recent-activity display, and a non-persistent demo-user cold-start flow.

- Persisted, fresh-process-loadable model artifacts for popularity, collaborative filtering, content-based filtering, the selected hybrid, and the cold-start router.
- A complete documentation package: README, final project report, reviewer/user guide, QA report, delivery checklist, evaluation report, architecture, data contract, API contract, technical decisions, handoff notes, and TODO record.

7. Challenges Faced

A number of issues were identified during development and resolved before final delivery:

#	Issue	Resolution
1	Initial MySQL authentication failed (error 1045).	The local account was corrected; the successful live run followed.
2	MySQL strict mode rejected CSV boolean strings (e.g. "False") for discount_applied.	The loader now converts them to 1/0, with a regression test.
3	Risk of accidental loads against an unintended database.	The loader reads .env itself and rejects missing or placeholder credentials.
4	Initial baseline evaluation counted previously seen holdout items as relevant even though they were correctly filtered from recommendations.	Evaluation now excludes seen holdout items from the relevance set, with a regression test; all reported metrics are from the corrected run.
5	Brute-force nearest-neighbor fitting was too slow on the full interaction data.	Replaced with sparse normalized item-matrix multiplication and top-neighbor pruning.
6	Running python src/collaborative.py failed because src was not importable from a direct file execution context.	Documented and standardized on python -m src.collaborative.
7	scikit-learn rejected NumPy's legacy np.matrix, and sparse conversion initially returned an object array for content scoring.	The content profile now explicitly uses CSR format with dense final score extraction; regression tests cover this behavior.
8	The initial hybrid grid search recomputed content candidates for every one of 21 candidate weights, causing long-running duplicate attempts.	Source candidates are now cached once per user; only the lightweight blending/ranking step repeats per weight.
9	Saved Joblib artifacts could not be loaded in a fresh process because earlier runs serialized model classes as __main__.	All model artifacts were regenerated via stable src.* imports with unchanged configuration and matching recorded metrics; fresh-process loading now passes.
10	The Phase 9 cold-start router artifact had the same __main__ serialization issue.	Regenerated through a stable src.cold_start import; the fresh-process router check now passes with unchanged results.
11	A CORS mismatch surfaced when Vite was served at 127.0.0.1 but the API allowed only localhost.	The API now accepts both explicit local origins, with a regression test and successful browser verification.
12	A combined activity-feature patch initially duplicated a Flask route/method, causing an endpoint collision.	API tests caught the collision immediately; the duplicate declarations were removed.

#	Issue	Resolution
13	The live integration test initially left .env database values in the pytest process environment, affecting a separate dotenv unit test.	The integration fixture now restores the original environment after execution.

7.1 Known Non-Blocking Issue

The final live integration run emitted twelve deprecation warnings originating from NumPy and Joblib serialization internals, related to a shape-setting behavior that a future NumPy version will reject. No application test failed because of this, and all saved artifacts load successfully in a fresh process; the warning is tracked as an upstream dependency-compatibility item rather than suppressed.

8. Possible Future Improvements

The following items are recorded as forward-looking maintenance and enhancement points; none of them has been implemented in the submitted version, and the current model selection and test-set boundaries remain frozen for this delivery.

- Monitor and address the NumPy/Joblib deprecation warning when dependency versions are next upgraded, including re-verifying fresh-process artifact loading after the upgrade.
- Evaluate price and promotion as structured content signals through a dedicated, documented experiment before considering their inclusion in the content or hybrid representation.
- Re-examine a content-inclusive hybrid blend only if a new validation experiment demonstrates an improvement over the current CF-only selection.
- Investigate a reliable product-image mapping for the frozen catalog, since none currently exists, before adding product imagery to the interface.
- Extend cold-start handling with a genuine post-deployment evaluation once real new-user interaction data becomes available, since no proxy relevance metric is used for new users today.

References

The following project documents were used as the source of truth for this documentation. All figures, decisions, and claims above are drawn directly from these documents.

- PROJECT_SPEC.md — Project specification, objective, frozen dataset, and technology constraints.
- ARCHITECTURE.md — System architecture, layer responsibilities, and request lifecycle.
- DATA_CONTRACT.md — Raw and processed data schema, validation rules, and generated outputs.
- TECH_DECISIONS.md — Phase-by-phase technical decisions and rationale.
- EVALUATION.md — Evaluation protocol, metrics, robustness audit, and cold-start routing evidence.
- API_CONTRACT.md — REST endpoint definitions, response shapes, and error contract.
- QA_REPORT.md — Automated test layers, final executed checks, and resolved defects.
- DELIVERY_CHECKLIST.md — Submission state, reviewer walkthrough, and accuracy guidelines.
- FINAL_PROJECT_REPORT.md — Executive summary, business problem, results, and limitations.
- USER_GUIDE.md — Reviewer and user guide for demonstrating the application.
- HANDOFF.md — Phase-by-phase completion log, dataset facts, resolved issues, and delivery status.
- MYSQL_SETUP.md — MySQL prerequisites, configuration, schema loading, and troubleshooting.

- [TODO.md](#) — Phase-by-phase task completion record.